

SOA Workshop - 01

Environment configuration

Goals

Prepare the environment for creating, compiling and deploying a web application

A. Set up the development environment

- Download and install the IDE **IntelliJ IDEA**
- Configure the EDI
- Discover the characteristics of **IntelliJ IDEA**

B. Install JDK and configure environment variables

C. Install the Maven tool

D. Setting up a JAVA project with JDK using IntelliJ

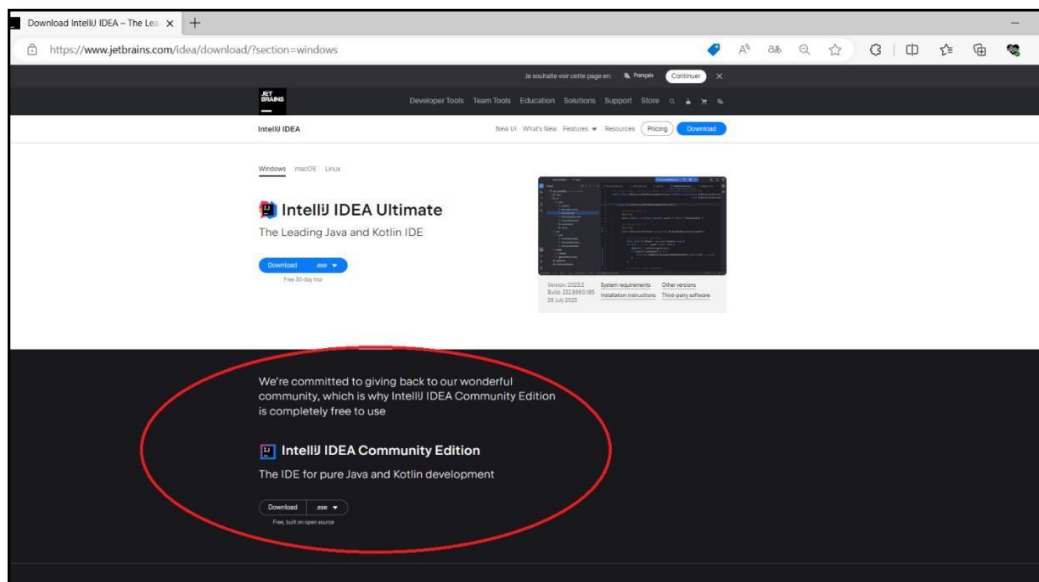
E. Using Basic Maven Commands (Lifecycle Phases)

F. Deploy a web application with Tomcat using Maven

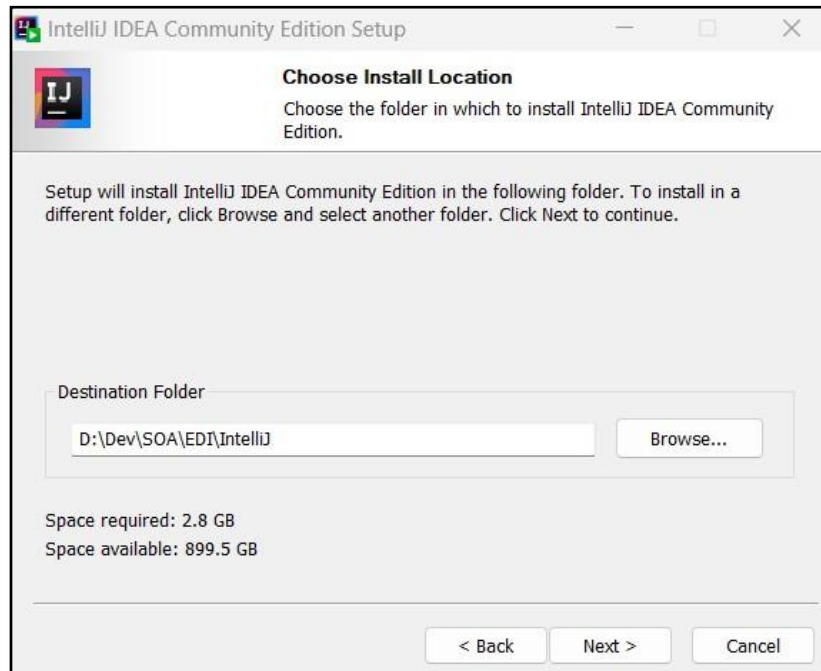
A. Set up the development environment

1.Download the (open source “.exe”) version of the IDE “**IntelliJ IDEA**» (**Community Edition**) through the following link:

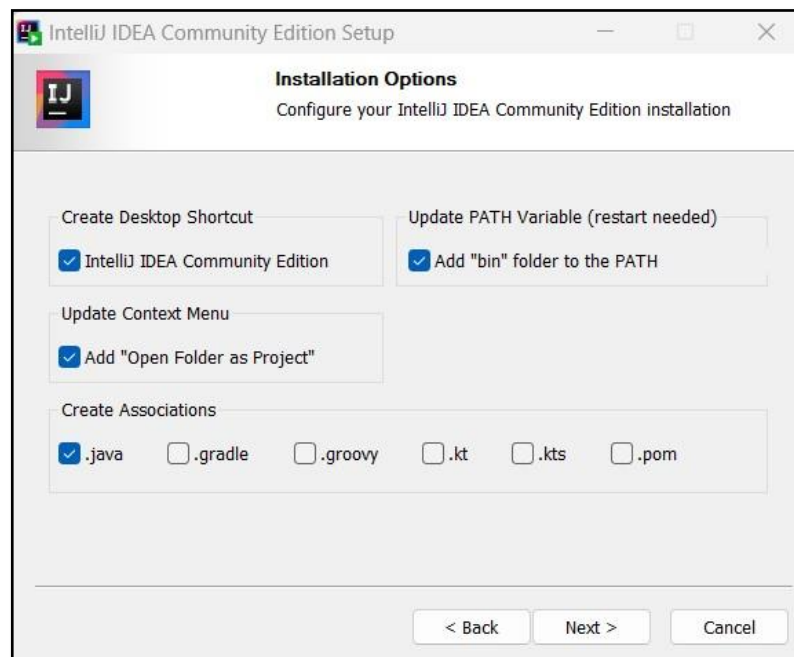
<https://www.jetbrains.com/idea/download/#section=windows>



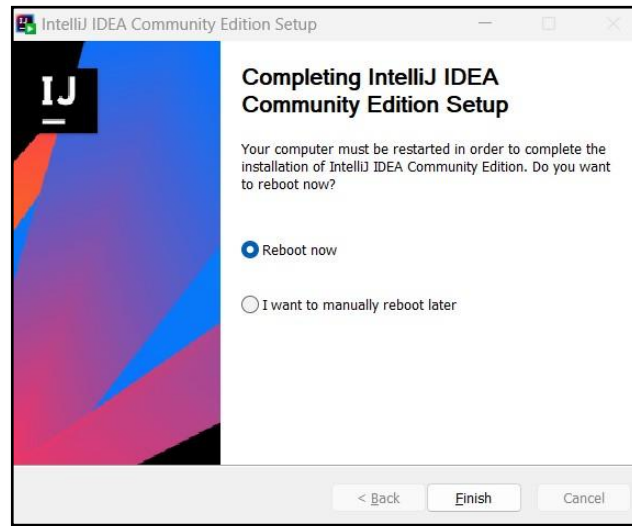
2.Double-click the downloaded executable (**ideaIC-2025.1**) to start the installation. Choose to install "**IntelliJ IDEA**»in the folder (**Destination Folder**) then click on “**Next**» :



3. Check the following options then click on **“Next»”** :



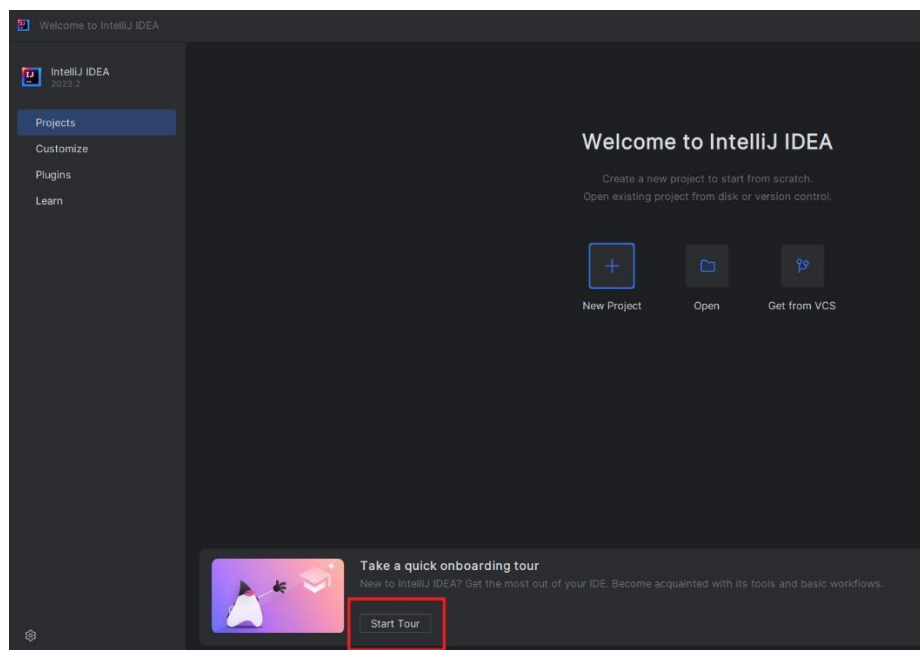
4. Complete the installation process, ask to restart the computer and finalize:



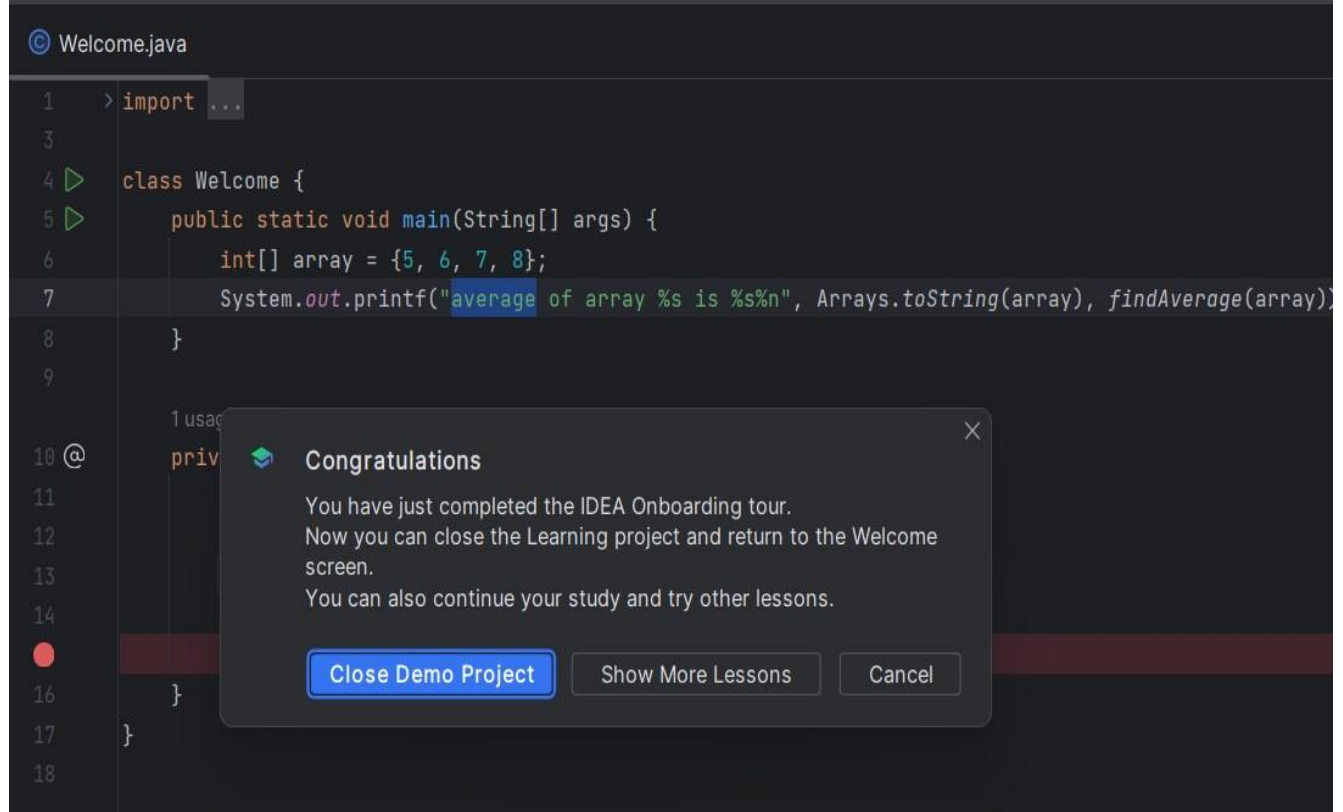
5. After reboot, Open "IntelliJ» by double-clicking the shortcut on the desktop

6. Before starting our first project with **IntelliJ**, go for a quick tour to discover the EDI. Click on "**Start tour**»

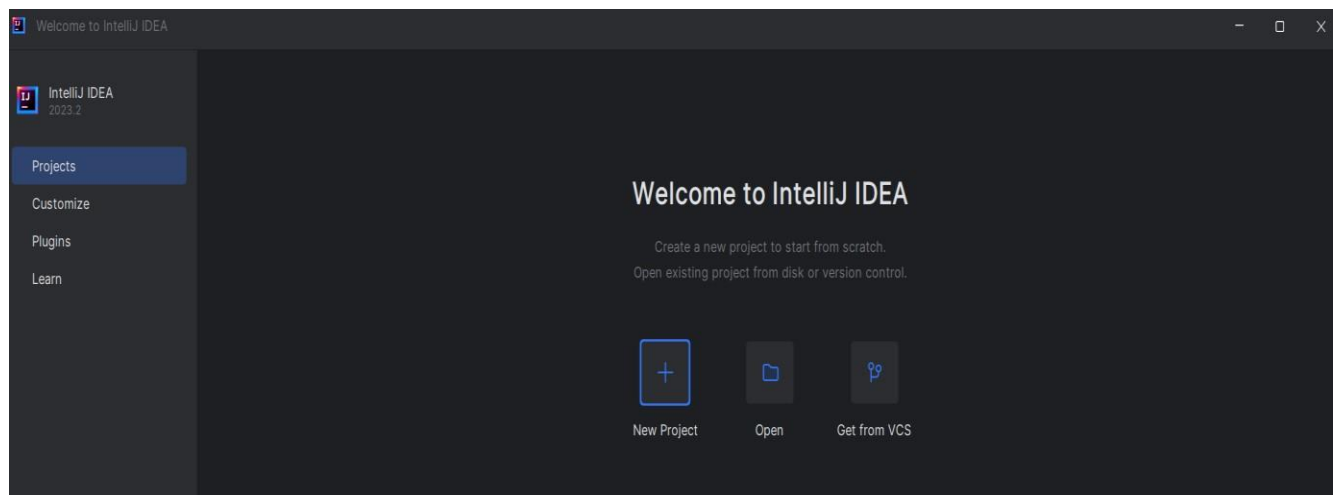
to begin this lesson on learning how to use the IDE:



b. Follow the instructions in the learning lesson (step by step) until you reach the end of this lesson:



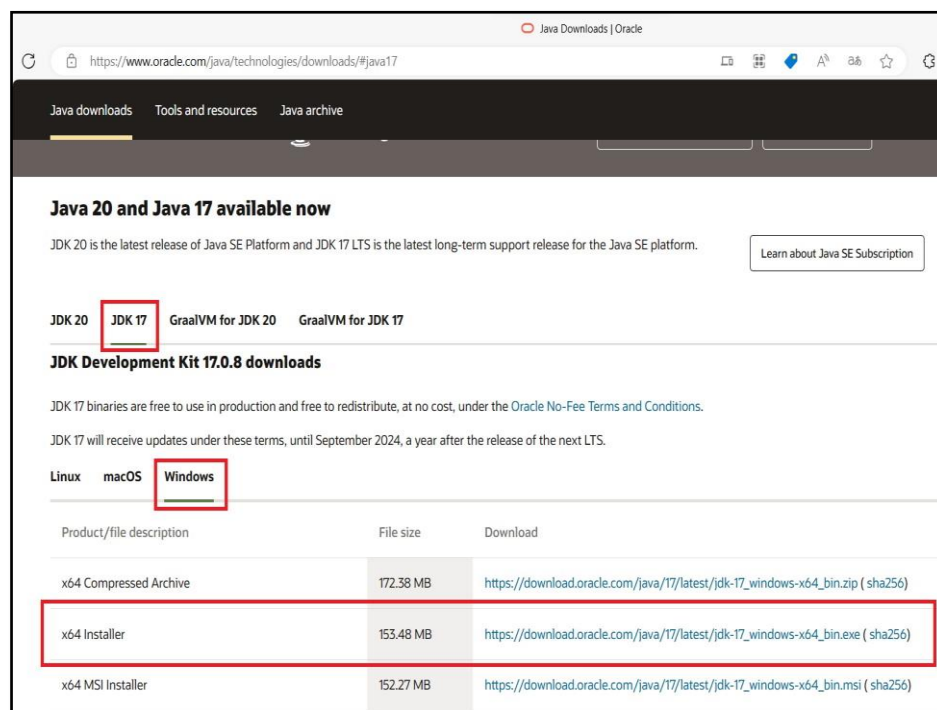
- c. You can continue to follow other lessons to better train yourself on the proper use of this IDE. For now, click on "**Close Demo Project**" to end this learning and go to the home screen:



B. Download and install "JDK17"

(Skip directly to section C. if JDK17 is already installed and configured)

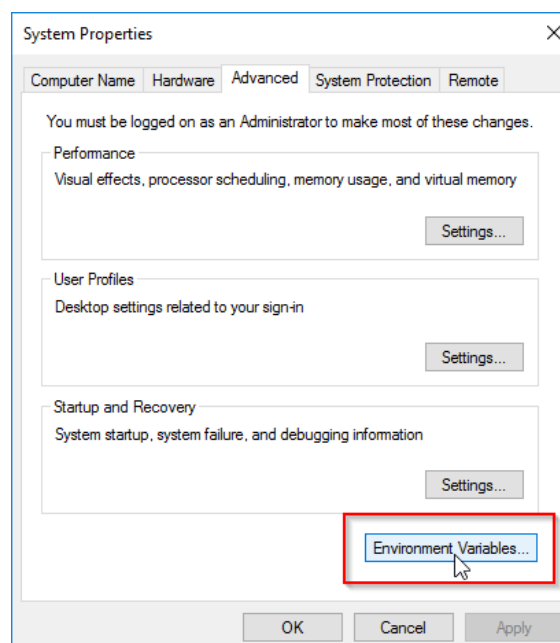
7. First, install **JDK17**. Here is the download link:



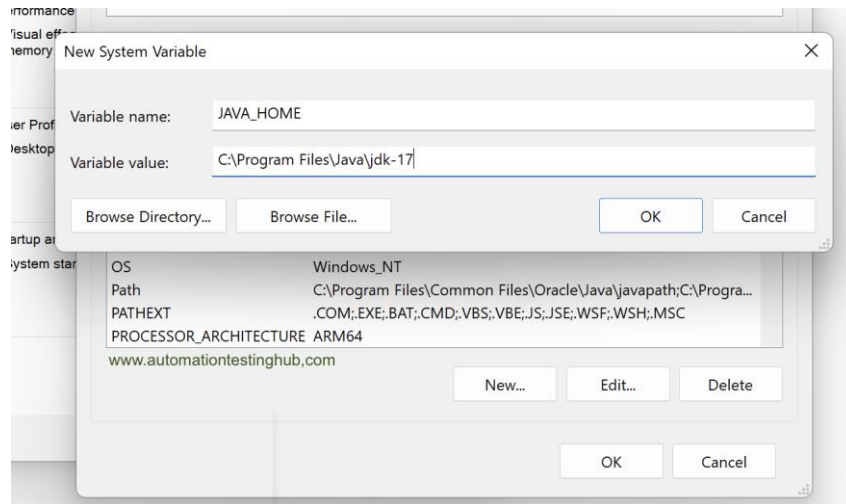
8. Once the JDK is installed, add the path to the folder "**bin**" of **JDK17** to the environment variable "**PATH**" (first). To do this, follow these steps:

a. open the system properties

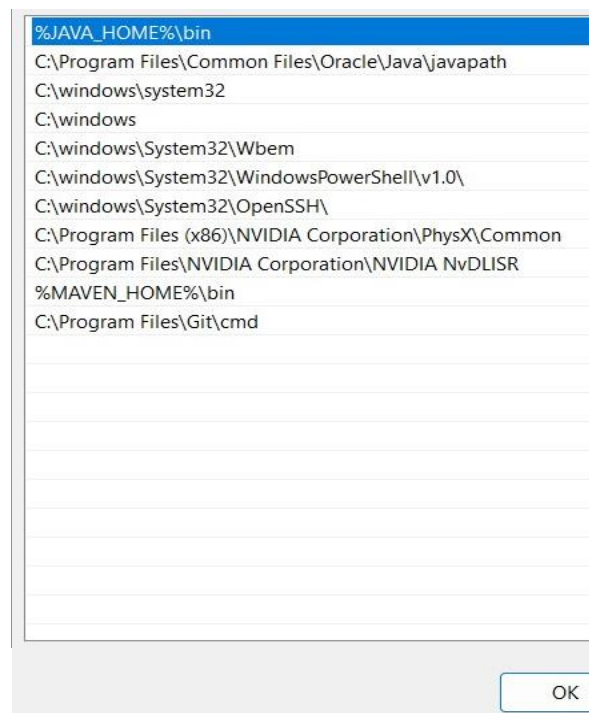
- Open the Control Panel.
- Click **System and Security**, then **System**
- Click **Advanced system settings** on the left
- Inside the System Properties window, click the **Environment Variables...** button.



- b. Add an environment variable "**System**» named **JAVA_HOME** having as value the installation path of JDK 17: "**C:\Program Files\Java\jdk-17**" : click on the button «**New**» to add the new variable)



- c. Double click on the system variable **Path** to add a new value "**%JAVA_HOME%\bin**» (which represents the path of the JDK 17 executables): Use the button «**Move up**» to place the new value at the top of the list. Finally, confirm the operation by pressing the button «**OK**».



C. Download and install the “Maven” tool

9. Here is the download link for "**Maven**» :

Downloading Apache Maven 3.9.11

Apache Maven 3.9.11 is the latest release: it is the recommended version for all users.

System Requirements

Java Development Kit (JDK)	Maven 3.9+ requires JDK 8 or above to execute. It still allows you to build against 1.3 and other JDK versions by using toolchains.
Memory	No minimum requirement
Disk	Approximately 10MB is required for the Maven installation itself. In addition to that, disk space will be used for your local Maven repository. The size of your local repository will vary depending on usage but expect at least 500MB.
Operating System	No minimum requirement. Start up scripts are included as shell scripts (tested on many Unix flavors) and Windows batch files.

Files

Maven is distributed in several formats for your convenience. Simply pick a ready-made binary distribution archive and follow the [installation instructions](#). Use a source archive if you intend to build Maven yourself.

In order to guard against corrupted downloads/installations, it is highly recommended to [verify the signature](#) of the release bundles against the public [KEYS](#) used by the Apache Maven developers.

	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.9.11-bin.tar.gz	apache-maven-3.9.11-bin.tar.gz.sha512	apache-maven-3.9.11-bin.tar.gz.asc
Binary zip archive	apache-maven-3.9.11-bin.zip	apache-maven-3.9.11-bin.zip.sha512	apache-maven-3.9.11-bin.zip.asc
Source tar.gz archive	apache-maven-3.9.11-src.tar.gz	apache-maven-3.9.11-src.tar.gz.sha512	apache-maven-3.9.11-src.tar.gz.asc
Source zip archive	apache-maven-3.9.11-src.zip	apache-maven-3.9.11-src.zip.sha512	apache-maven-3.9.11-src.zip.asc

10. Choose the type "**Binary zip archive**" (version 3.9.11: currently latest version)

11. Unzip the file "**apache-maven-3.9.4-bin.zip**" in the same folder.

12. Add a system environment variable corresponding to the path: **MAVEN_HOME**

13. Add the value "**%MAVEN_HOME%\bin**" into the path variable

14. Open a window «**Command Prompt**» and test the `mvn -version` configuration of "**Maven**» through the DOS command (If everything is fine, the versions of **Maven**, of **JAVA** and of **Windows** displayed in the will be DOS console).

```
C:\Users\HP>mvn -version
Apache Maven 3.9.4 (dfbb324ad4a7c8fb0bf182e6d91b0ae20e3d2dd9)
Maven home: D:\Dev\Outils\Maven\apache-maven-3.9.4
Java version: 17.0.8, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-17
Default locale: fr_FR, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

15. Now run the command `mvn clean`. This command generates an error due to the absence of a project of type which requires the existence of a file "**pom.xml** » (see creating a Maven project in the next section)

```
C:\Users\HP>mvn clean
[INFO] Scanning for projects...
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 0.111 s
[INFO] Finished at: 2023-08-29T12:33:44+01:00
[INFO] -----
[ERROR] The goal you specified requires a project to execute but there is no POM in this directory (C:\Users\HP). Please
verify you invoked Maven from the correct directory. -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MissingProjectException
```

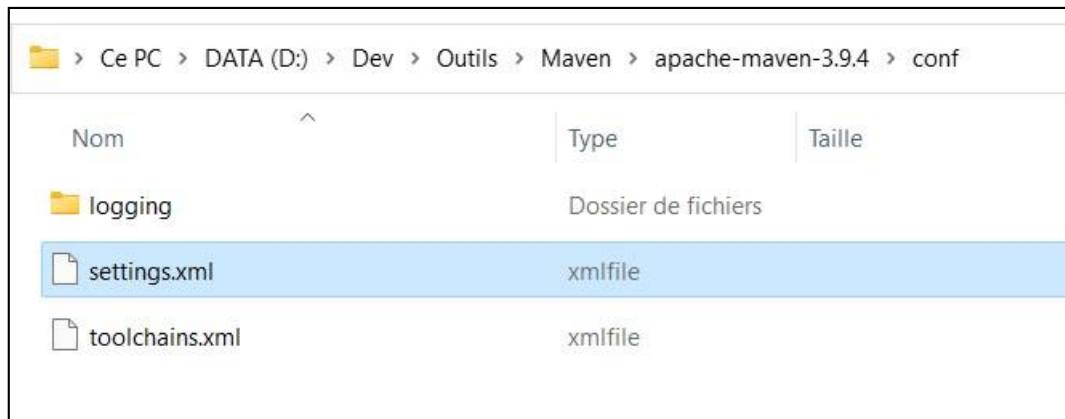
16. Remark the generation of a case "**C:\Users\HP\.m2\repository**" This file consists of the local deposit of **Maven**. («**C:\Users\HP**» is the root folder of the current user):



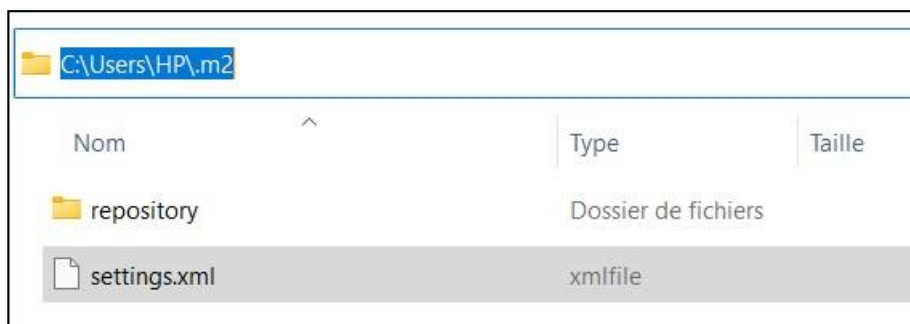
Nom	Type	Taille
antlr	Dossier de fichiers	
aopalliance	Dossier de fichiers	
asm	Dossier de fichiers	
backport-util-concurrent	Dossier de fichiers	
ch	Dossier de fichiers	
classworlds	Dossier de fichiers	
com	Dossier de fichiers	
commons-codec	Dossier de fichiers	
commons-io	Dossier de fichiers	
commons-lang	Dossier de fichiers	
io	Dossier de fichiers	
jakarta	Dossier de fichiers	
javax	Dossier de fichiers	
junit	Dossier de fichiers	
net	Dossier de fichiers	
org	Dossier de fichiers	

17. It is possible to change the default location of the local repository from **Maven** by following the instructions below:

- Copy the file "**settings.xml**» which exists in the file "**..\apache-maven-3.9.11\conf**".

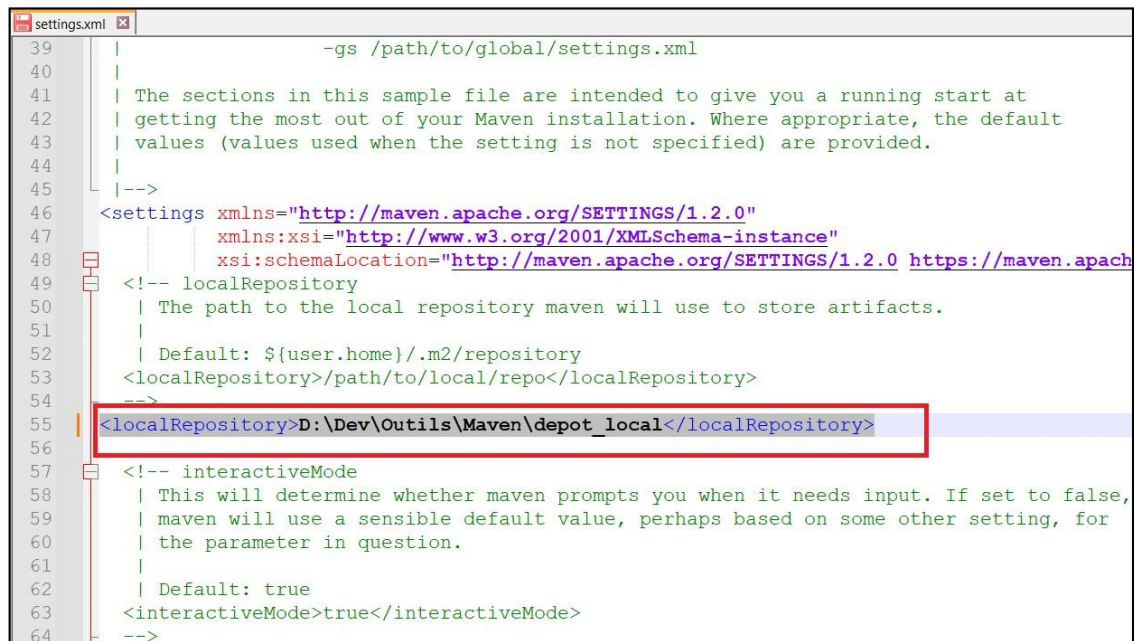


b) Paste it into the folder "C:\Users\HP\.m2» :



c) Create the subfolder "local_depot"

d) Open the new file "**settings.xml**» (copied as ".m2") and add the selected line:



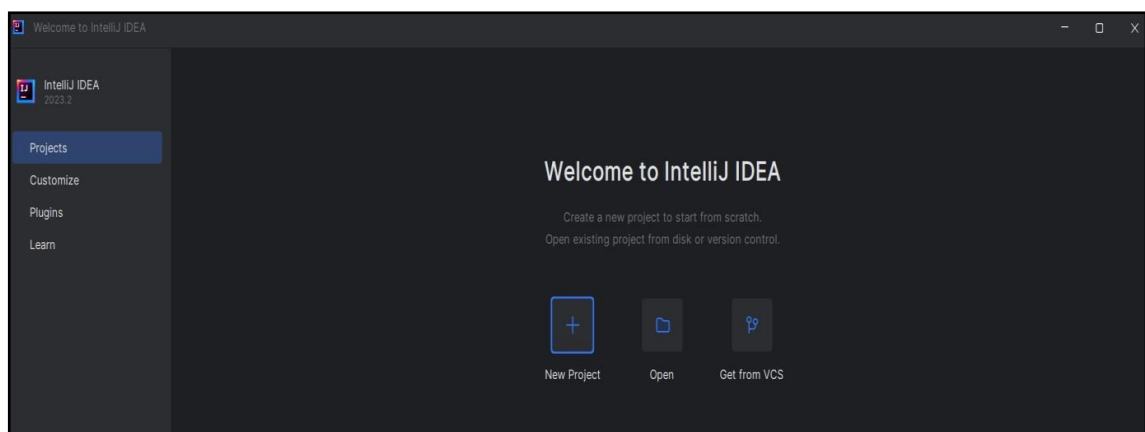
So the local Maven repository is now:

D:\Dev\Tools\Maven\local_depot

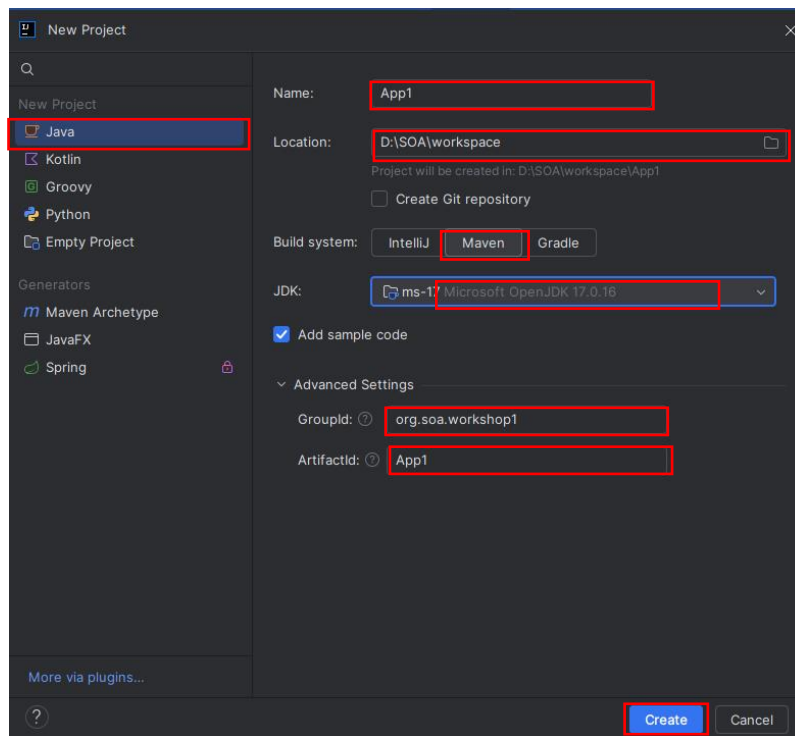
From now on, all downloads to be made by "Maven» (dependencies, plugins, etc.) will be stored locally in this location.

D. Create a first JAVA project with IntelliJ of MAVEN type

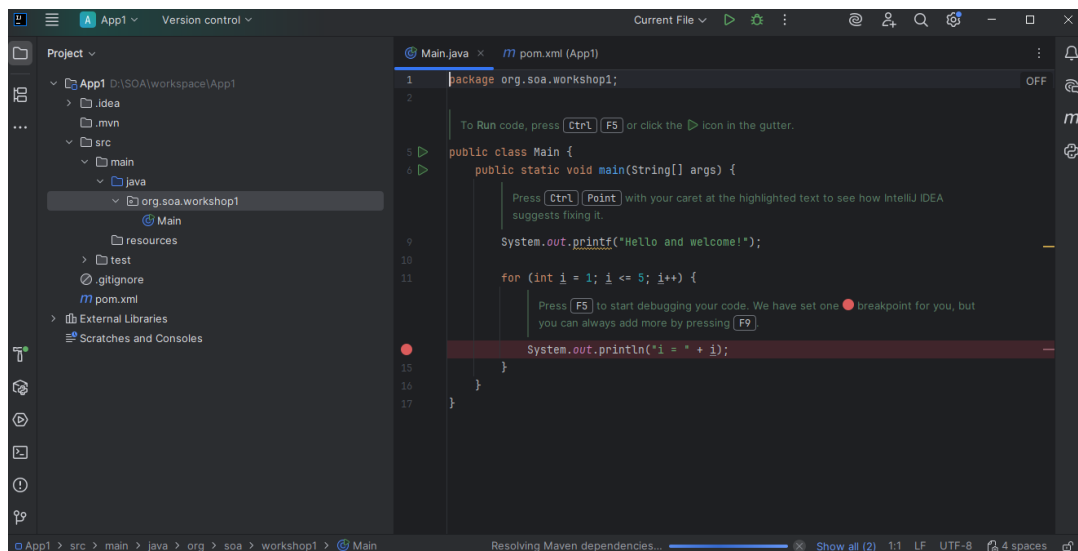
19. Let's go back to IntelliJ to create a first project by choosing the option "New Project» :



20. Specify project settings "Maven» as follows and start creation:



21.Start the execution of the class "Main» auto generated:

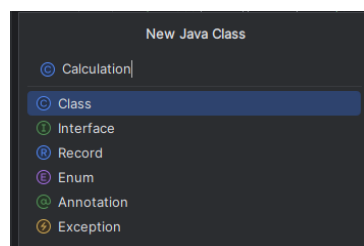
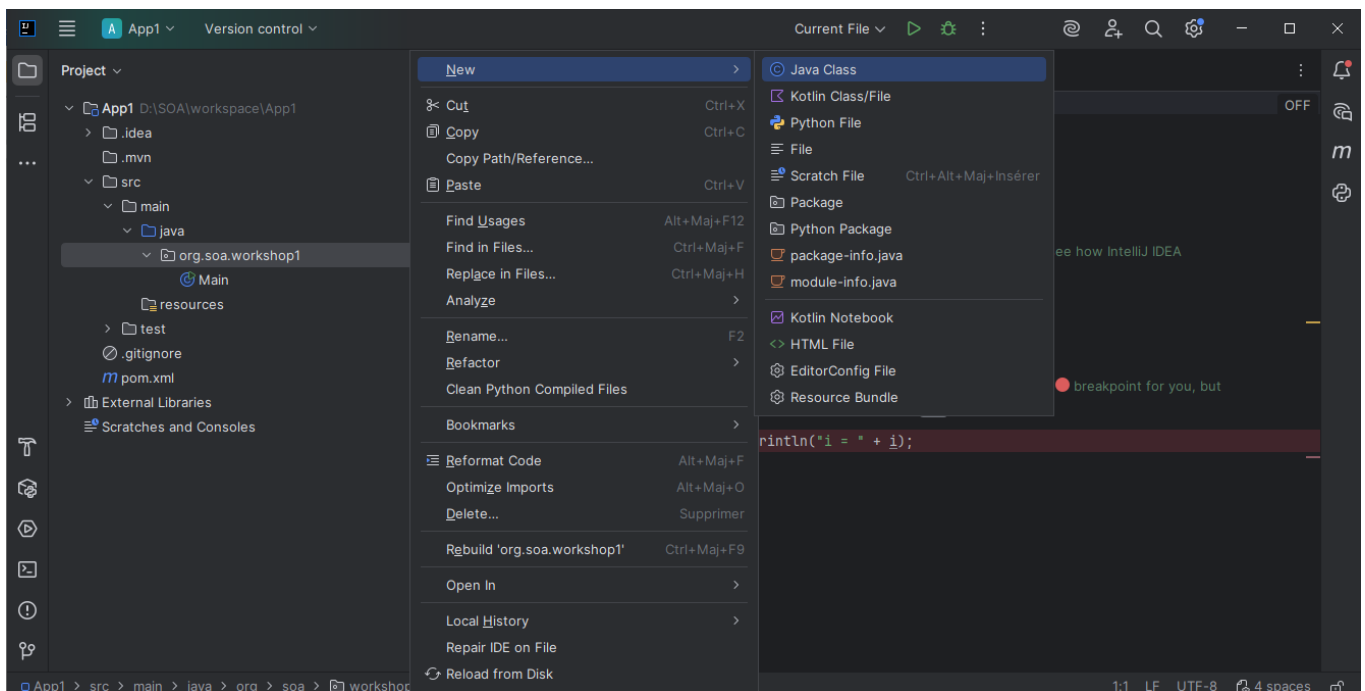


22.View the result in the "Run»at the very bottom:



```
"C:\Program Files\Java\jdk-17\bin\java.exe" -javaagent:D:\Dev\SOA\EDI\IntelliJ\lib\idea_rt.jar=5051
Hello and welcome! i = 1
i = 2
i = 3
i = 4
i = 5
Process finished with exit code 0
```

23. Add a class «**Calculation**» in the package «**org.soa.workshop1**» :



24. Enter the following code for the class "**Calculation**» :

```
package org.soa.workshop1;

public class Calculation {
    public double sum(double a, double b){
        return a+b;
    }
}
```

```

    }
    public double produt(double a, double b){
        return a*b;
    }
}

```

25. Change the startup class code "**Main**" to test the functioning of the new methods "**sum**" And "**product**"

26. Start the execution and notice that the execution is not associated with arguments (no arguments are passed to the program "**Main**):

```

package org.soa.workshop1;

public class Main {
    public static void main(String[] args) {

        if(args.length<2)
        {
            System.out.println("Please enter two numbers...");
            System.exit(0);
        }
        double a = Double.parseDouble(args[0]);
        double b = Double.parseDouble(args[1]);
        Calculation cm =new Calculation();
        double s =cm.sum(a, b);
        double p = cm.produt(a, b);
        System.out.println("The sum of "+a+"And "+b+"is: "+s);
        System.out.println("The product of "+a+"And "+b+"is: "+p);

    }
}

```

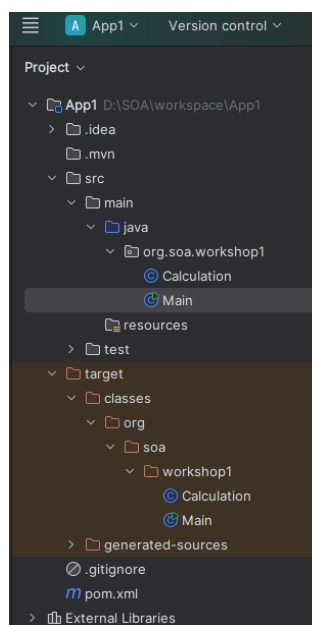
```

C:\Users\USER\.jdk\ms-17.0.16\bin\java.exe "-javaagent:D:\SOA\IntelliJ IDEA Community Edition 2025.2
Please enter two numbers...

Process finished with exit code 0

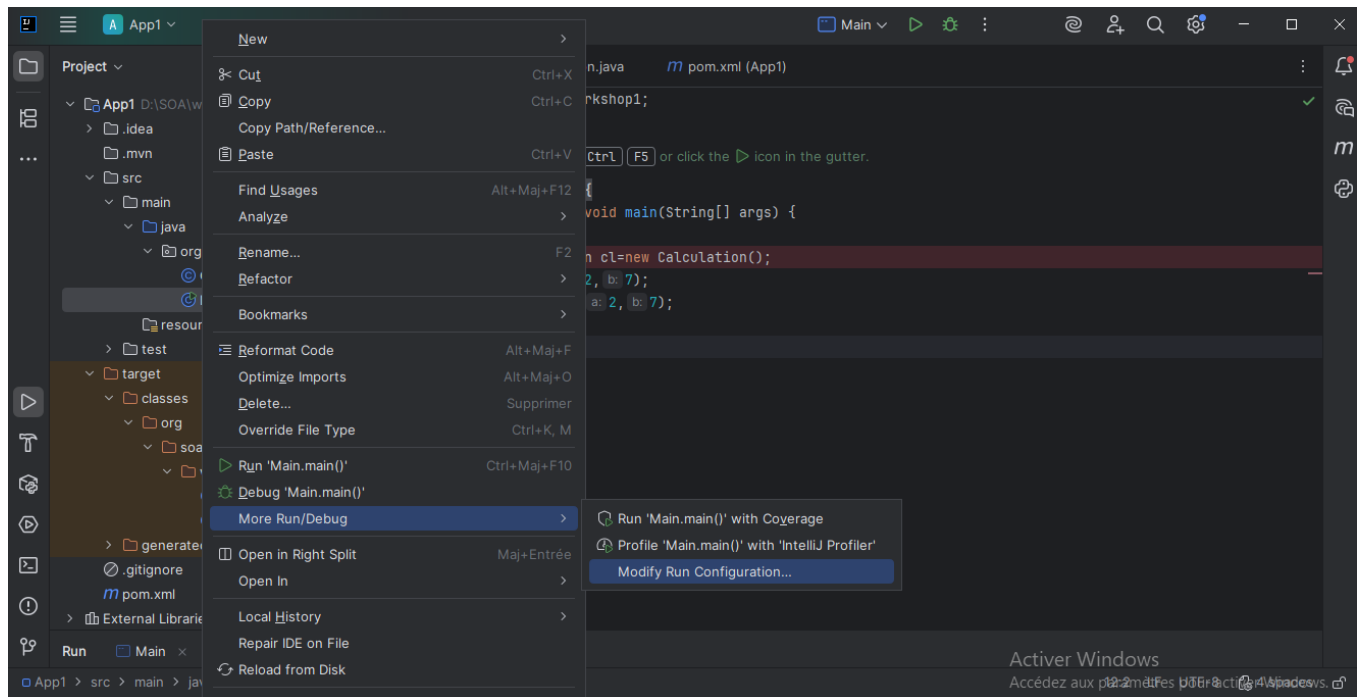
```

Note, also, the generation of a subfolder "**target/classes**" which groups the precompiled files (.class):

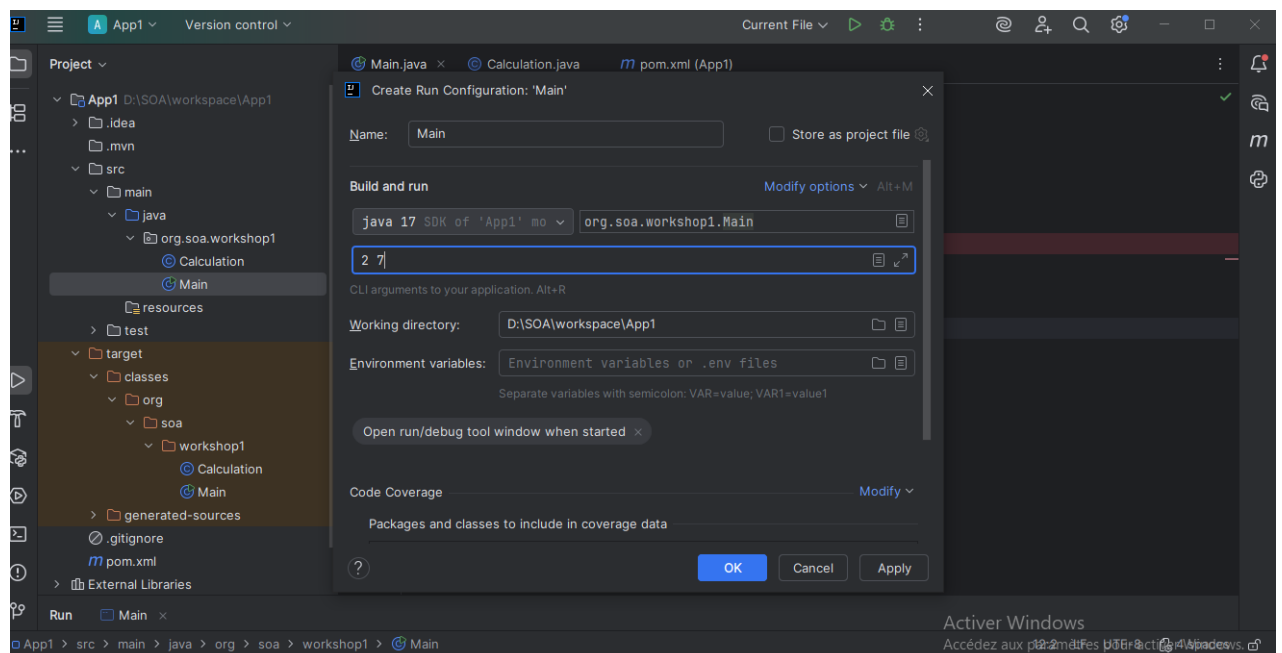


27.To pass arguments at runtime:

- a) Select the class “Main» and with the right mouse button, choose the command “ **Modify Run Configuration...**»



- b. Designate, as an example, two arguments: **2** and **7** separated by space as follows:



- c.Restart the execution and view the result:

E. Use basic Maven commands to manage the project lifecycle

In the 'default' lifecycle of a Maven project, the used phases are:

- **clean:** deletes all files **Java .class** previously compiled
- **validate:** checks the configurations of a maven project
- **compile:** compile the source code
- **test:** compiles and runs unit tests
- **package:** assembles the compiled code into a deliverable
- **install:** share the deliverable for other projects on the same computer
- **deploy:** publish the deliverable for other projects in the remote "repository"
- The phases are executed in a **sequential** so that a phase depends on the previous phase.
- For example, the user launching the test phase (**mvn test**) will involve the prior launch by maven of the phases "**validate**" And "**compile**".

28. It is worth noting that it is possible to automate the testing procedure with unit tests using the library **JUnit** And **Maven** To do this:

a). Add under "**src/test/java**" a package "**org.soa.workshop1**".

b). Add in the package "**org.soa.workshop1**" a test class "**ClaculTest**" having the following code:

```

Package org.soa.workshop1;

import static org.junit.Assert.*;

import org.junit.BeforeClass;

import org.junit.Test;

public class CalculTest{
    //Declare a reference to the Calculation class

    private static Calculation job;

    @BeforeClass
    public static void setUpBeforeClass() throws Exception {
        //Instantiate the CalculMetier class
        job=new Calculation();
    }

    @Test

    public void testSum()
    { assertTrue(job.sum(5,2) ==7);
    }

    @Test

    public void testProduct()
    { assertTrue(job.product(5,2) ==10); }
}

```

- The method "**setUp()BeforeClass**» is executed before all tests of the class: It is annotated with **@BeforeClass**
- All test methods are annotated by **@Test**
- The method **assertTrue(Boolean condition)** is a static method of the class "**org.junit.Assert**" and indicates that the test is correct if the condition is correct.

c. Notice that the code presents a set of **errors** due to the absence of the library **JUnit**:

- It is therefore necessary to declare in the file "**pom.xml**» the dependence of **JUnit** that Maven brings it to our project.
- To do this, add the code **in bold** below to the file "**pom.xml**» inside the **< project>** tag:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi
:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.soa.workshop1</groupId>
  <artifactId>App1</artifactId> <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.source>17</maven.compiler.source> <maven.compiler.target>17</
maven.compiler.target> <project.build.sourceEncoding>UTF-8</ project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId> <artifactId>
junit</artifactId> <version>4.13.2</ version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>

```

d. **Save** the modifications and **load** MAVEN changes by pressing the button located at the top right in the file "**pom.xml**" :

e. Notice the correction of errors in the class "**CalculTest**" and adding two libraries related to JUnit's MAVEN dependency:

- **Manage the project lifecycle in the "Terminal"**

29. Open the "**Terminal**" In **IntelliJ**: A command prompt window opens, positioned under our project folder "**App1**" :

- ✓ Type the command **clean** to delete all files **.class** recently compiled (clean project): **mvn clean** This will delete the subfolder "**target**" :
- ✓ Type the command **validate** to check that the project configuration is correct (no missing elements...) **mvn validate**

```

PS D:\SOA\workspace\FirstApp> mvn validate
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.soa.workshop1:FirstApp >-----
[INFO] Building FirstApp 1.0-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time:  1.073 s
[INFO] Finished at: 2025-09-24T08:44:30+01:00
[INFO]

```

- ✓ Type the command **compile** to compile the source files (not the test files) and then generate the files **.class** under a subfolder **"/target/classes"** : **mvn compile**

```

Terminal  Local x + v
[INFO] --- resources:3.3.1:resources (default-resources) @ premier ---
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ premier ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 2 source files with javac [debug target 17] to target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time:  1.866 s

```

- ✓ Type the command **test** to compile and run the unit test class: **mvn test**

Notice that the test is completed successfully (without errors) and a precompiled class is generated for the test class under **"target/test-classes"** in a package **"org.soa.workshop1"** :

c. Edit class **"Calculation"** to have incorrect operation of the methods **"sum"** And **"product"** Take the following example:

```

package org.soa.workshop1;

public class Calculation
{
    double public sum(double a, double b) {

        return a - b; // instead of a+b
    }

    double public product(double a, double b) {

        return a / b; // instead of a*b
    }
}

```

d. Restart compilation and testing: No problem at the compilation level. However, at the test phase, two errors were triggered:

```

[INFO]
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR]   CalculMetierTest.testProduit:27
[ERROR]   CalculMetierTest.testSomme:21
[INFO]
[ERROR] Tests run: 2, Failures: 2, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----

```

e. Resume the correct version of the class "**Calculation**" and redo the compilation and test.

- ✓ Type the command **package** to archive the project and package it in a distributable format (JAR, WAR, etc.). Note the creation of an archive package "**App1-1.0SNAPSHOT.jar**" directly in the subfolder «**target**» : **mvn package**
- ✓ Try to run the program through the newly created archive file using the following command:

Java **-jar. \target\App1-1.0-SNAPSHOT.jar**

An error is raised to indicate that it is not possible to detect a startup class with a main method:

- ✓ To fix this problem, edit the file "**pom.xml**" and add a configuration for the plugin «**jar**» to specify the main class (startup class) as follows:

```
<dependencies>
  <dependency>
    <groupId>junit</groupId> <artifactId>
      junit</artifactId> <version>4.13.2</ version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <artifactId>maven-jar-plugin</artifactId>
      <version>3.4.1</version> <configuration>

        <archive>
          <manifest>
            <mainClass>org.soa.workshop1.Main</mainClass> </ manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

d.Carry out the following actions in succession:

- Save changes
- Start loading Maven changes
- Redo the archiving phase (**mvn package** enter the following command:

Java **-jar. \target\App1-1.0-SNAPSHOT.jar**

```
PS D:\Dev\SOA\workspace\premier> java -jar .\target\premier-1.0-SNAPSHOT.jar
Veuillez saisir deux nombres..
```

✓ Pass, as an example, the two arguments **5** and **6**:

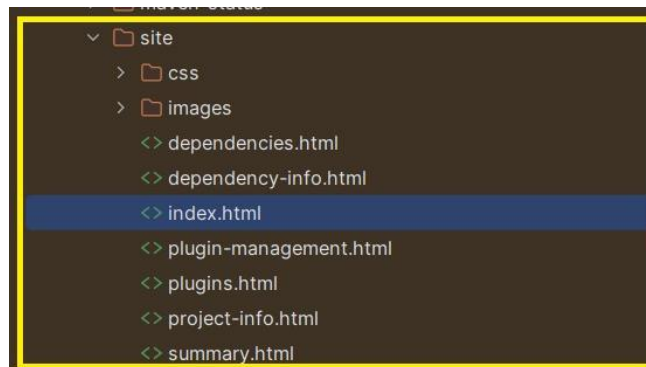
✓ Type the command **install** to deliver the project archive to the local Maven folder: **mvn install**

```
[INFO] --- install:3.1.1:install (default-install) @ premier ---
[INFO] Installing D:\Dev\SOA\workspace\premier\pom.xml to D:\Dev\Utils\Maven\depot_local\org\soa\tp1\premier\1.0-SNAPSHOT\premier-1.0-SNAPSHOT.pom
[INFO] Installing D:\Dev\SOA\workspace\premier\target\premier-1.0-SNAPSHOT.jar to D:\Dev\Utils\Maven\depot_local\org\soa\tp1\premier\1.0-SNAPSHOT\premier-1.0-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.205 s
```

Notice the delivery of the project archive to the local Maven repository:

✓ Type the command **site** to generate a site for the project. This site also includes the project reports that were configured in the "**pom.xml**"» **mvn site**

Remark the creation of a subfolder "**/target/site**"» Who brings together HTML pages describing the project:



Open the index.html file with a web browser to view the home page of the project site. **App1** :

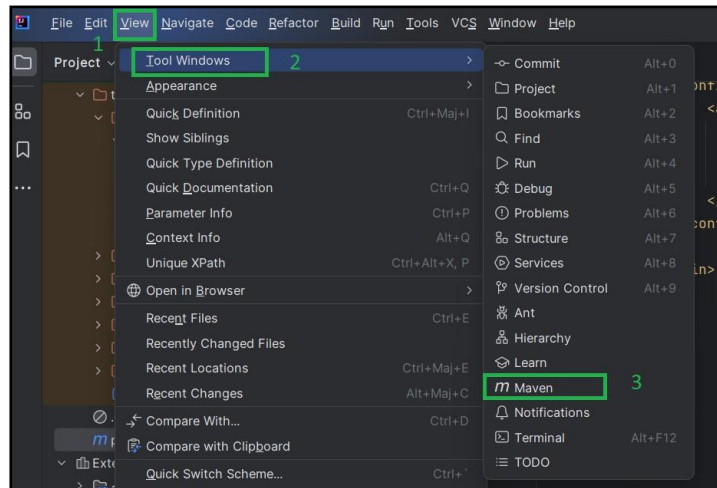
The navigation pane presents links to describe dependencies, plugins, and project information.

• Manage the project lifecycle in Graphically

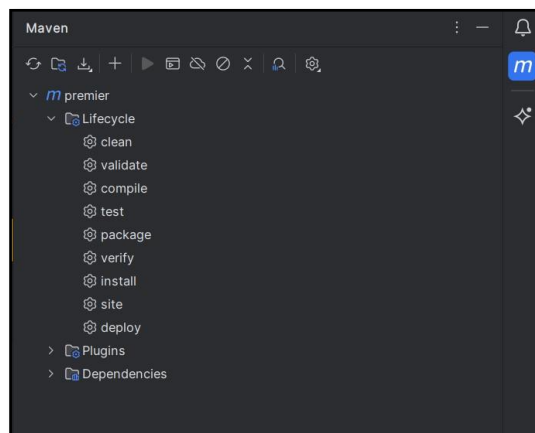
37. All these Maven commands are accessible graphically through the IDE **IntelliJ**:

- Access the menu "**View**"»
- Choose "**Windows Tool**"»

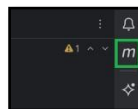
- Choose "**Maven**» :



This displays a toolbox to use the desired Maven commands:



These commands are also accessible through the button on the top right:

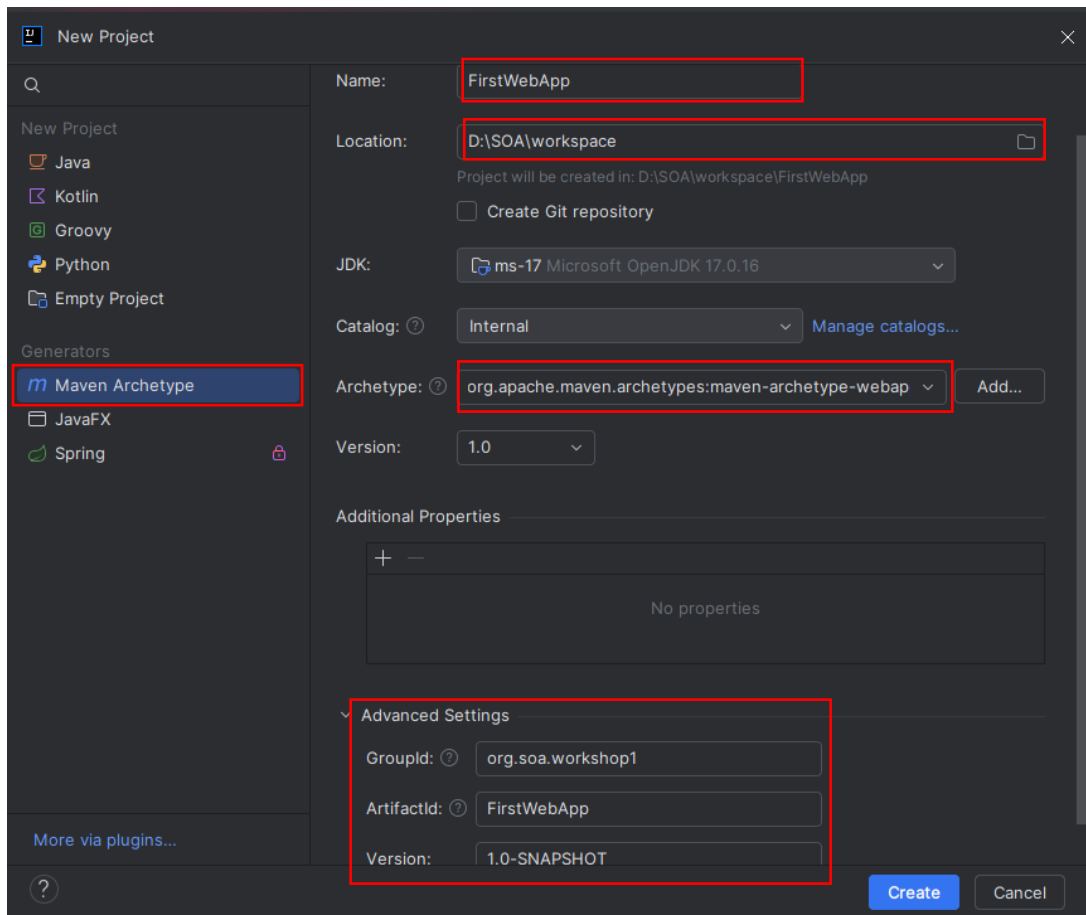


F. Deploy a web application with Tomcat using Maven

38. Create a web application with a Maven template (archetype) and specify the project settings:

- **groupId**: org.soa.workshop1
- **artifactId**: webProject
- **version**: 0.0.1-SNAPSHOT

- **package:org.soa.workshop1**(same value as "groupId")

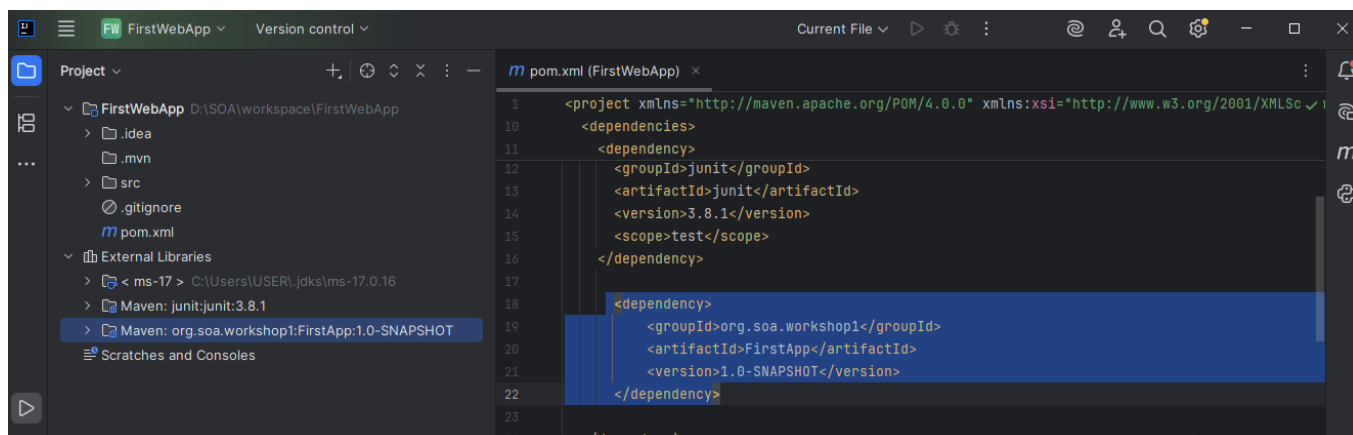


39. This new project depends on our first project "**FirstApp**" (i.e. uses its classes). To do this, open the file "**pom.xml**" of the project "**FirstWebProject**" to add the following dependency inside the two tags **<dependencies>** and **</dependencies>** following the dependence of "**junit**":

```
<dependency>
  <groupId>org.soa.workshop1</groupId>
  <artifactId>FirstApp</artifactId>
  <version>1.0-SNAPSHOT</version>
</dependency>
```

NB: Don't forget to save and synchronize Maven after each modification of the pom.xml

40. Notice the addition of the new library:



41. Edit the file "index.jsp" of the project and which exists under «src/main/webapp» as follows:

```

<%@page import="org.soa.workshop1.Calculation" contentType="text/html;
    charset=UTF-8"%> <% double a=
0.0; double b=0.0; double res=0.0; String action = request.getParameter("action");

if (action != null) {

    a = Double.parseDouble(request.getParameter("a"));
    b = Double.parseDouble(request.getParameter("b"));
    Calculation cl= new Calculation();
    if (action.equals("sum")){
        res=cl.sum(a,b);
    } else { res=cl.produit(a,b); }
}
%>
<html>
<body>
<form action ="index.jsp"method ="post"> <table>

<tr>
<td> To: </td> <td> <input type ="text" name ="a" value ="<%=a%>"/> </td> <td> B: </td> <td> <input type
="text" name ="b" value ="<%=b%>"/></td>
</tr> <tr>
<td> <input type ="submit" name ="action" value ="sum"/></td> <td> <input type ="submit" name
="action" value ="product"/> </td>
</tr>
<tr> <td> Result: </td> <td> <%=res%></td> </tr> </table>

</form>
</body>
</html>

```

42. Run the Maven command "install» :

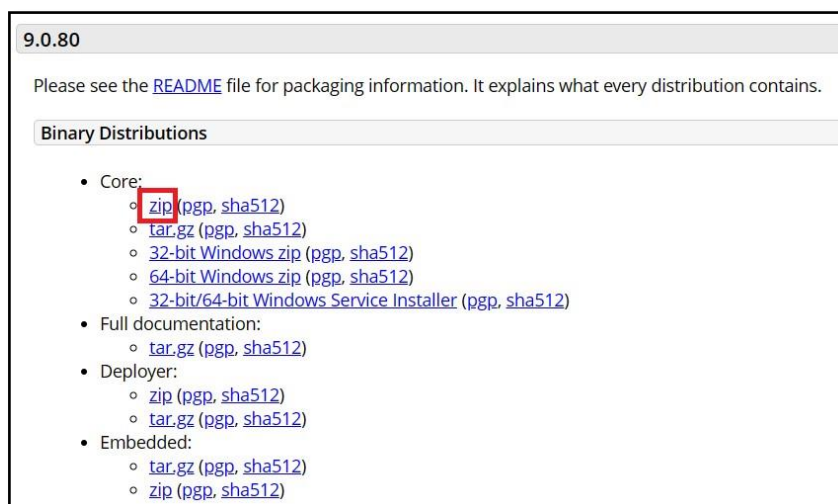


Notice the delivery of the project archive (in WAR format) to the local Maven repository:

Ce PC > Disque local (D:) > SOA > depot_local > org > soa > workshop1 > FirstWebApp > 1.0-SNAPSHOT				
Nom	Modifié le	Type	Taille	
_remote.repositories	24/09/2025 06:38	Fichier REPOSITOR...	1 Ko	
FirstWebApp-1.0-SNAPSHOT.pom	24/09/2025 06:27	Fichier POM	1 Ko	
FirstWebApp-1.0-SNAPSHOT.war	24/09/2025 06:38	Fichier WAR	4 Ko	
maven-metadata-local	24/09/2025 06:38	Microsoft Edge H...	1 Ko	

43. Now let's configure the web server. **Tomcat**» (version 9). Access the following URL:

<https://tomcat.apache.org/download-90.cgi>



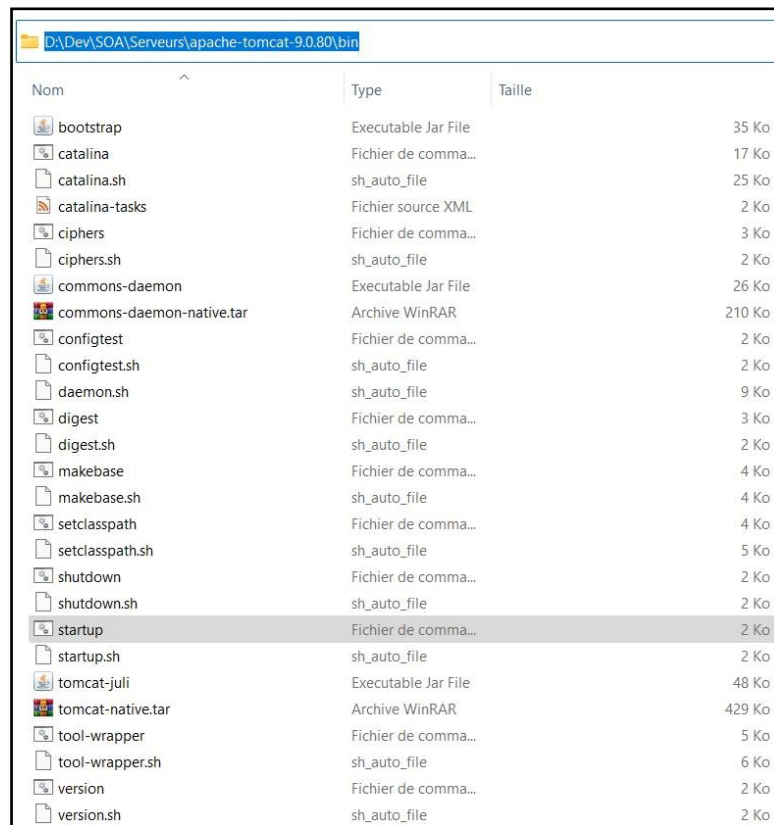
44. Download the archive (zip) of the **Tomcat**» in the folder:

D:\SOA\Tools

45. Extract the archive from the **Tomcat**» in the folder:

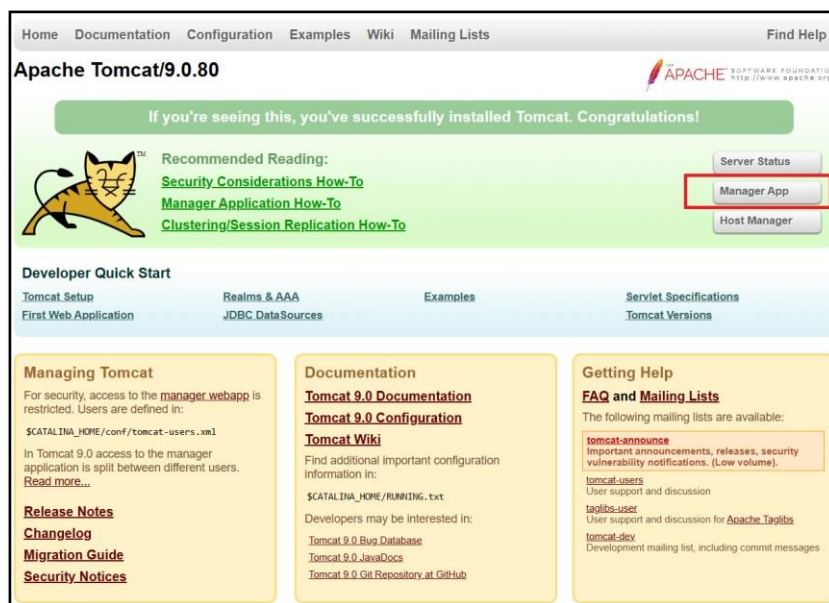
D:\SOA\Servers

46. Access the folder "**bin**" of Tomcat and launch program "**startup.bat**" :

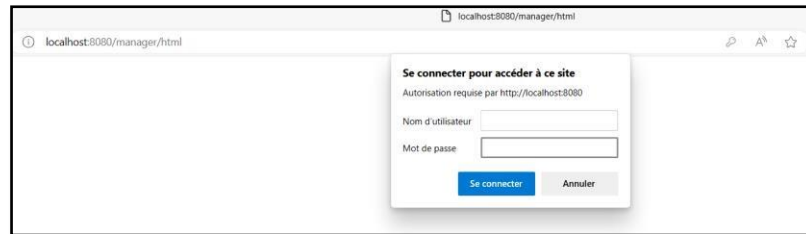


47. Access the server through the web browser using the link: **http://localhost:8080**

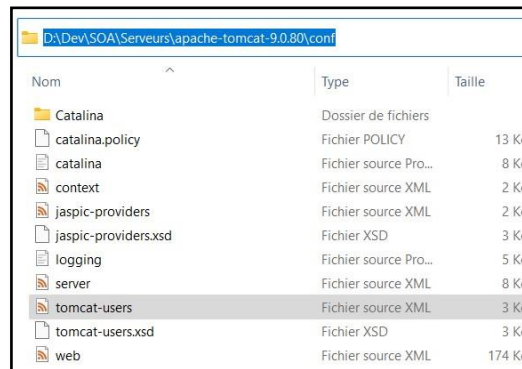
48. Here is the Tomcat home page:



49. Access the administration panel «**Manager App**». Tomcat requires authentication with a username and password:



50.To do this, access the subfolder “**conf**»from Tomcat and edit the file“**tomcatusers.xml**» :



51.Add, between the two tags **<tomcat-users>** and **</tomcatusers>**, the following instructions to define a user (**admin,admin**) having the necessary rights to manage web applications:

```
<role rolename="manager-gui"/>

<role rolename="admin-gui"/>

<role rolename="manager-script"/>

<user username="admin" password="admin" roles="manager-gui,admin-gui, manager-script"/>
```

52.Save the file then Stop the server «**Tomcat**»by clicking on the file "**shutdown.bat**»(in the subfolder "**bin**").

53.Restart «**Tomcat**» (by clicking on the file**startup.bat**) and re-access the tomcat administration web interface to authenticate with the two parameters (“**admin**» , « **admin**”)and manage deployed applications:

localhost:8080/manager/html

Message : ok

Gestionnaire

[Lister les applications](#) [Aide HTML Gestionnaire](#) [Aide Gestionnaire](#)

Applications

Chemin	Version	Nom d'affichage	Fonctionnelle	Sessions	Commandes
/	Aucun spécifié	Welcome to Tomcat	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/docs	Aucun spécifié	Tomcat Documentation	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/examples	Aucun spécifié	Servlet and JSP Examples	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/host-manager	Aucun spécifié	Tomcat Host Manager Application	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/manager	Aucun spécifié	Tomcat Manager Application	true	1	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes

Deployer

Emplacement du répertoire ou fichier WAR de déploiement sur le serveur

Chemin de contexte (requis):

Version (pour les déploiements en parallèle):

URL du fichier XML de configuration:

URL vers WAR ou répertoire:

Deployer

54. Access the “Deploy \WAR file to deploy» to deploy the web application which is already delivered in the local repository.

Deployer

Emplacement du répertoire ou fichier WAR de déploiement sur le serveur

Chemin de contexte (requis):

Version (pour les déploiements en parallèle):

URL du fichier XML de configuration:

URL vers WAR ou répertoire:

Deployer

Fichier WAR à déployer

Choisir le fichier WAR à téléverser Aucun fichier choisi

55. Select the project WAR file “FirstWebProject» already installed in the local Maven repository:

56. Then click on “Deploy» to have the deployed web project in the list of applications taken into account by the Tomcat server:

Applications					
Chemin	Version	Nom d'affichage	Fonctionnelle	Sessions	Commandes
/	Aucun spécifié	Welcome to Tomcat	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/docs	Aucun spécifié	Tomcat Documentation	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/examples	Aucun spécifié	Servlet and JSP Examples	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/host-manager	Aucun spécifié	Tomcat Host Manager Application	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/manager	Aucun spécifié	Tomcat Manager Application	true	1	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes
/webProject-0.0.1-SNAPSHOT	Aucun spécifié	Archetype Created Web Application	true	0	Démarrer Arrêter Recharger Retirer Expirer les sessions inactives depuis ≥ 30 minutes

57. So the WEB project is deployed, you just have to click on the corresponding link to start its execution.

58. Exercise

Make the necessary changes so that the page **index.jsp** also displays a third button " **Division**» to carry out an operation **a / b**